IMPLEMENTATION GUIDE

Django Multi-Vendor Marketplace Roadmap

A practical sequence for adding authentication, vendors, product ownership, personal carts, inventory, relational orders, and secure checkout to the existing e-commerce backend.

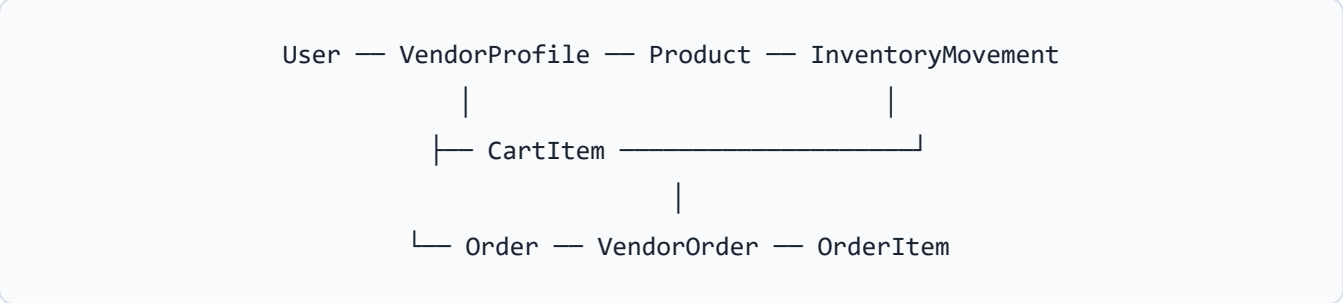
> **Recommended first milestone:** authenticated APIs, vendor profiles, vendor-owned products with basic stock, and customer-specific carts.

Project: **ecommerce_backend**
Framework: Django + Django REST Framework
Prepared: September 2026

1. Starting point and target structure

The current project already has a custom user model, products, delivery options, carts, and orders. Before it becomes multi-user, three existing behaviors must change:

- `CartItem.product` is currently one-to-one, so a product can appear in only one cart across the database.
- Cart and order queries currently read every user's rows.
- Orders store their products as JSON instead of relational order-item records.



> **Protect existing data:** add new required foreign keys as nullable first, populate them in a data migration, verify the records, and only then make the fields required.

2. Build order

Phase	Feature	Completion signal
1	Authentication and API permissions	Anonymous users cannot access carts or orders.
2	Vendor profiles	A user can apply and an admin can approve the store.
3	Product ownership and stock	Vendors manage only their own products.
4	Customer-specific carts	Two users can add the same product independently.
5	Relational orders	Orders use immutable item and price snapshots.
6	Atomic checkout	Checkout safely creates an order and decreases stock.
7	Vendor order splitting	Each vendor sees and fulfils only its part.
8	Inventory history	Every stock change has a reason and actor.
9	Ownership permissions	Cross-user and cross-vendor access is denied.
10	Tests and documentation	Critical isolation and checkout cases pass.

PHASE 1 Authentication and API permissions

Use Django session authentication initially because the project already has a Django login page. Add `rest_framework` to `INSTALLED_APPS`, then configure:

```
REST_FRAMEWORK = {
    "DEFAULT_AUTHENTICATION_CLASSES": [
        "rest_framework.authentication.SessionAuthentication",
    ],
}
```

Apply permissions by endpoint:

- Public product list and product detail: `AllowAny`.
- Cart, orders, checkout, and payment summary: `IsAuthenticated`.
- Vendor management: authenticated plus a custom vendor permission.

Add account endpoints:

```
POST /api/auth/register
POST /api/auth/login
POST /api/auth/logout
GET  /api/auth/me
```

Session-authenticated POST, PUT, PATCH, and DELETE requests must include a valid CSRF token.

PHASE 2 Vendor profiles

Create a dedicated app and register it in `INSTALLED_APPS`:

```
python manage.py startapp vendors
```

```
class VendorProfile(models.Model):
    class Status(models.TextChoices):
        PENDING = "pending", "Pending"
        APPROVED = "approved", "Approved"
        SUSPENDED = "suspended", "Suspended"

    owner = models.OneToOneField(
        settings.AUTH_USER_MODEL,
        on_delete=models.CASCADE,
        related_name="vendor_profile",
    )
    store_name = models.CharField(max_length=255)
    description = models.TextField(blank=True)
    status = models.CharField(
        max_length=10,
        choices=Status.choices,
        default=Status.PENDING,
    )
```

When an administrator approves a vendor, set the account role to `vendor`. A vendor should still be allowed to purchase products.

```
POST  /api/vendors/apply
GET   /api/vendors/me
PATCH /api/vendors/me
```

PHASE 3 Product ownership and basic inventory

Add ownership, stock, publication status, and a vendor SKU to `products/models.py`:

```
vendor = models.ForeignKey(
    VendorProfile,
    on_delete=models.PROTECT,
    related_name="products",
)
stock_quantity = models.PositiveIntegerField(default=0)
is_active = models.BooleanField(default=True)
sku = models.CharField(max_length=100, unique=True)
```

Because 42 products already exist, use staged migrations:

1. Add `vendor` with `null=True`.
2. Create a vendor profile for the existing `rahul` user.
3. Assign the existing products to that profile with a data migration.
4. Verify every product has a vendor.
5. Change `vendor` to `null=False`.

Never accept a vendor ID from the create-product request. Assign ownership on the server:

```
serializer.save(vendor=request.user.vendor_profile)
```

Vendor product queries must filter by owner:

```
Product.objects.filter(vendor__owner=request.user)
```

PHASE 4 Customer-specific carts

Replace the current one-to-one product relationship with a normal foreign key and add the customer:

```
class CartItem(models.Model):
    user = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.CASCADE,
        related_name="cart_items",
    )
    product = models.ForeignKey(
        Product,
        on_delete=models.CASCADE,
        related_name="cart_items",
    )
    quantity = models.PositiveIntegerField(default=1)
    delivery_option = models.ForeignKey(
        DeliveryOption,
        on_delete=models.PROTECT,
    )

    class Meta:
        constraints = [
            models.UniqueConstraint(
                fields=["user", "product"],
                name="unique_product_per_user_cart",
            )
        ]
```

Scope every cart query to the authenticated user:

```
permission_classes = [IsAuthenticated]

def get_queryset(self):
    return CartItem.objects.filter(user=self.request.user)

def perform_create(self, serializer):
    serializer.save(user=self.request.user)
```

When merging quantities, look up both the user and product:

```
CartItem.objects.get(user=request.user, product=product)
```

Reject quantities below one, above the configured cart limit, or above available stock.

PHASE 5 Relational orders and item snapshots

Replace the JSON-only order design with relational records. Store the purchased name and price so later product edits do not change old orders.

```
class Order(models.Model):
    class Status(models.TextChoices):
        PENDING = "pending", "Pending"
        CONFIRMED = "confirmed", "Confirmed"
        CANCELLED = "cancelled", "Cancelled"
        COMPLETED = "completed", "Completed"

    customer = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.PROTECT,
        related_name="orders",
    )
    status = models.CharField(
        max_length=20,
        choices=Status.choices,
        default=Status.PENDING,
    )
    total_cost_cents = models.PositiveIntegerField()
    created_at = models.DateTimeField(auto_now_add=True)
```

```
class OrderItem(models.Model):
    order = models.ForeignKey(
        Order,
        on_delete=models.CASCADE,
        related_name="items",
    )
    product = models.ForeignKey(
        Product,
        on_delete=models.SET_NULL,
        null=True,
    )
    product_name = models.CharField(max_length=255)
    unit_price_cents = models.PositiveIntegerField()
    quantity = models.PositiveIntegerField()
```

Keep the old `products` JSON field temporarily. Convert the three existing orders with a `RunPython` data migration, verify the new rows, and then remove the JSON field.

PHASE 6 Atomic checkout

Move checkout calculations out of model defaults and into `orders_and_paymentsummary/services.py`. The service should:

1. Load only the authenticated user's cart.
2. Reject an empty cart.
3. Recheck prices and stock on the server.
4. Create the order and item snapshots.
5. Decrease inventory.
6. Delete only that user's cart items.
7. Return the completed order.

```
from django.db import transaction

with transaction.atomic():
    # Validate stock, create order rows, decrease stock,
    # and clear only request.user's cart.
    ...
```

Concurrency: SQLite does not provide effective row locking through `select_for_update()`. Move to PostgreSQL before relying on concurrent stock reservations in production.

PHASE 7 Split checkout by vendor

```
class VendorOrder(models.Model):
    order = models.ForeignKey(
        Order,
        on_delete=models.CASCADE,
        related_name="vendor_orders",
    )
    vendor = models.ForeignKey(
        VendorProfile,
        on_delete=models.PROTECT,
    )
    status = models.CharField(max_length=20, default="pending")
    subtotal_cents = models.PositiveIntegerField()
```

During checkout, group cart items by vendor and create one `VendorOrder` for each group. This allows each vendor to ship, cancel, or refund its own portion independently.

PHASE 8 Inventory history

```
class InventoryMovement(models.Model):
    product = models.ForeignKey(Product, on_delete=models.CASCADE)
    quantity_change = models.IntegerField()
    reason = models.CharField(max_length=50)
    created_by = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.PROTECT,
    )
    created_at = models.DateTimeField(auto_now_add=True)
```

Change	Reason
+20	Stock received
-2	Order placed
+2	Order cancelled
-1	Damaged item

Provide a vendor stock-adjustment endpoint that updates the product and creates an inventory movement in the same transaction.

PHASE 9 Ownership permissions

Create custom permissions:

```
IsApprovedVendor
IsProductOwner
IsVendorOrderOwner
```

Permissions and queryset filtering must work together. Required behavior:

- Customer A cannot view Customer B's cart or orders.
- Vendor A cannot modify Vendor B's products.
- Vendor A cannot view Vendor B's sales.
- Customers cannot use vendor management endpoints.
- Suspended vendors cannot create or update listings.

PHASE 10 Tests and API documentation

Write tests for business rules where a failure could expose another user's data or corrupt stock:

- ☐ Two customers can add the same product to separate carts.
- ☐ A customer sees only their own cart and orders.
- ☐ A vendor can modify only their own products.
- ☐ Cart quantity cannot exceed available stock.
- ☐ Checkout decreases stock and records item snapshots.
- ☐ Checkout clears only the current customer's cart.
- ☐ A failed checkout leaves stock and cart unchanged.
- ☐ A checkout with two vendors creates two vendor orders.

Update `documentation.md` after each phase so request fields, permissions, status codes, and example responses stay accurate.

3. Commands to run after each model phase

```
.\venv\Scripts\python.exe manage.py makemigrations
.\venv\Scripts\python.exe manage.py migrate
.\venv\Scripts\python.exe manage.py check
.\venv\Scripts\python.exe manage.py test
```

Inspect every generated migration before applying it, especially migrations that add required foreign keys or transform existing order data.

4. First milestone checklist

- ☐ Register, log in, log out, and return the current user.
- ☐ Require login for carts, orders, checkout, and payment summaries.
- ☐ Create and approve vendor profiles.
- ☐ Assign every product to a vendor.
- ☐ Add SKU, stock quantity, and active status to products.

- [ ] Allow vendors to manage only their own products.
- [ ] Associate every cart item with a user.
- [ ] Allow separate users to add the same product.
- [ ] Reject cart quantities that exceed stock.
- [ ] Update and test the API documentation.

5. Reference documentation

- Django REST Framework: Authentication
- Django REST Framework: Permissions
- Django: Model constraints
- Django: Database transactions
- Django: Migrations